StackRAG

A LLM and RAG powered AI Coding Assistant as a VS Code Extension

Software Project Lab 3 Technical Report, 2025

Submitted by
Shifat Jahan Shifa
BSSE Roll No: 1301
Institute of Information Technology
University of Dhaka

Supervised by
Dr. Kazi Muheymin-Us-Sakib
Professor
Institute of Information Technology
University of Dhaka



Institute of Information Technology
University of Dhaka
Submission Date: 13.01.2026

Letter of Transmittal

January 13, 2026
BSSE 4th Year Exam Committee
Institute of Information Technology
University of Dhaka

Subject: Technical Report Submission - StackRAG: A LLM and RAG powered AI Coding Assistant as a VS Code Extension.

Dear Sir,
I am submitting the technical report for my Software Project Lab 3, *StackRAG: A LLM and RAG powered AI Coding Assistant AS VS Code Extension.* This report comprehensively covers the project's technical details, including design, implementation, and testing phases. While I've tried to do my best, I welcome your feedback for improvement.

Thank you for your consideration.

Sincerely,
Shifat Jahan Shifa
BSSE-1301
Institute of Information Technology
University of Dhaka

13.01.2026
—----------
Supervisor's Signature

Acknowledgment

I would like to express my sincere gratitude to the Institute of Information Technology, University of Dhaka, for the opportunity to work on the "StackRAG: A LLM and RAG powered AI Coding Assistant AS VS Code Extension" project. This journey has been both rewarding and enlightening. A special thanks to my supervisor, **Dr. Kazi Muheymin-Us-Sakib,** Professor at the Institute of Information Technology, whose guidance and encouragement were invaluable throughout the project. His insights helped shape the project's direction and fueled my commitment to excellence. I also appreciate my classmates' camaraderie and valuable feedback, which added depth to this experience and helped refine the project.

Abstract

Modern software development frequently involves searching for solutions to programming errors, bugs, and implementation challenges, often requiring developers to manually browse large volumes of online resources such as Stack Overflow. This process is time-consuming and error-prone, especially in large or fast-paced projects where quick and reliable assistance is essential. Traditional search-based approaches provide fragmented information without sufficient contextual understanding of the developer's code.

To address these challenges, we propose **StackRAG**, a retrieval-augmented generation (RAG)–based system designed to provide context-aware programming assistance by leveraging Stack Overflow knowledge. StackRAG retrieves relevant question–answer pairs based on the developer's query and code context, and then uses large language models to generate concise, accurate, and explainable responses. By grounding the generated answers in retrieved, real-world developer discussions, StackRAG reduces hallucinations and improves the reliability of code-related suggestions.

The proposed system enhances the developer experience by delivering targeted explanations, practical solutions, and code-level guidance directly within the development workflow. StackRAG aims to reduce debugging time, improve problem-solving efficiency, and support maintainable software development by combining trusted community knowledge with the reasoning capabilities of modern language models.

Table of Contents

Letter of Transmittal **1**
Acknowledgment **2**
Abstract **3**
1. Introduction **6**
1.1 Motivation 6
1.2 Project Description 9
1.3 Project Scope 9
1.4 Timeline 10
1.5 Purpose of Document 10
2. Project Description **10**
2.1 Quality Function Deployment 10
2.2.1 Normal Requirements 11
2.2.2 Expected Requirements 11
2.2.3 Exciting Requirements 11
These features go beyond user expectations, providing unique value. Their presence can greatly enhance user satisfaction 11
2.2 Usage Scenario 11
3. Scenario-Based Modeling **13**
3.1 Use Case Diagram 13
3.1.1 Level 0: 13
3.1.1 Level 1: 14
3.1.1 Level 1.1: 16
3.1.1 Level 1.2: 16
3.1.1 Level 1.4: 17
3.1.1 Level 1.4.3: 18
4. Data Based Modeling **19**
5. Class-Based Modeling **19**
5.1 Analysis Classes 20
5.2 Class Cards 20
5.3 CRC Diagram 25
6. Architectural Design **26**
6.1 Architectural Context Diagram 26
6.2 Archetypes 27
6.3 Top Level Components 30
6.4 Mapping Requirements to Software Architecture 30
6.5 Deployment Diagram 31
7. Preliminary Test Plan **31**
7.1 High-Level Description of Testing Goals 31
7.2 Test Cases 32
Conclusion **35**
Reference **36**

List of Figures

Figure 1: Screenshot of the responses of the question....06
Figure 2: Survey questions....07
Figure 3: Survey question-1 07
Figure 4: Survey question-2 08
Figure 5: Survey question-3 08
Figure 6: Survey question-4 08
Figure 7: Project Timeline....10
Figure 8: Level 0 (StackRAG) 13
Figure 9: Level 1 (Modules of StackRAG) 14
Figure 10: level 1.1 (knowledge base system)....16
Figure 11: level 1.2 (codebase indexing system) 17
Figure 12: level 1.4 (answer generation system)....17
Figure 13: level 1.4.3 (SO posts retriever system) 18
Figure 14: CRC Diagram of StackRAG....25
Figure 15: Architectural Context Diagram 27
Figure 16: Layered Architecture....27
Figure 17: Archetypes 28
Figure 18: Top Level Components....30
Figure 19: Deployment diagram....31

List of Tables

Table-1: Schema table....19
Table-1: CRC card (Models)....20
Table-3: CRC card (KeywordExtractor)....21
Table-4: CRC card (Retriever)....21
Table-5: CRC card (CustomEnsembleRetriever)....21
Table-6: CRC card(AnswerGeneration)....22
Table-7: CRC card(SOPostsFetcher)....22
Table-8: CRC card(Ingestion)....22
Table-9: CRC card(BuildAndSaveBM25)....23
Table-10: CRC card(JSChunker)....23
Table-11: CRC card(PYChunker)....23
Table-12: CRC card(CodeIngestion)....24
Table-13: CRC card(CodebaseIndexManager)....24
Table-14: CRC card(VSCodeExtensionManag....25
Table-15: Requirements Mapping with components....30
Table-16: Test Plan -1 32
Table-17: Test Plan -2 32
Table-18: Test Plan -3 33
Table-19: Test Plan -4 33
Table-20: Test Plan -5 33
Table-21: Test Plan -6 33
Table-22: Test Plan -7 34
Table-23: Test Plan -8 34
Table-24: Test Plan -9 34
Table-25: Test Plan -10 35
Table-26: Test Plan -11 35
Table-27: Test Plan -12 35

1. Introduction

Stack Overflow has long been a trusted platform for developers to find community-verified solutions. Nowadays, with the rise of Large Language Models like ChatGPT, Gemini etc, many developers also rely on them for quick answers. However, each approach has its drawbacks. Searching through Stack Overflow threads can be time-consuming and requires filtering through multiple posts to find a clear solution. On the other hand, Large Language Models can sometimes produce answers that seem correct but are actually inaccurate or unreliable.

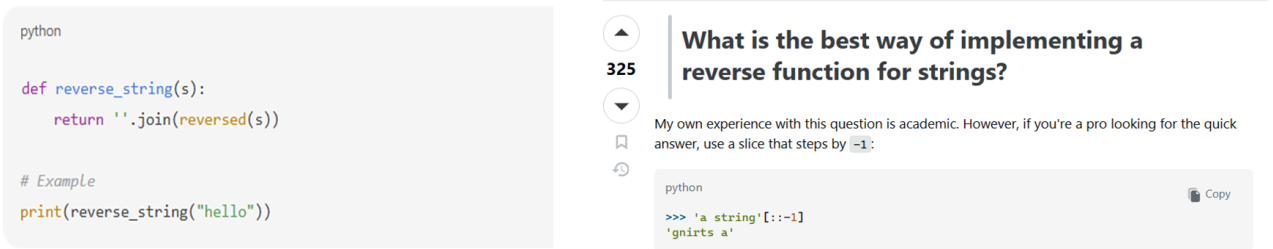
Our proposed tool **StackRAG** searches through Stack Overflow posts related to python and javascript languages. It fetches the accepted answers of questions and builds a question answer knowledge base. In the runtime, developers ask questions. The tool searches through the knowledge base to find the question-answer pairs relevant to the query. Then it passes the retrieved information along with the codebase context to Large Language Models to generate responses.

1.1 Motivation

One day, while I was practicing different string manipulation operations, a question came to my mind. The question is "how to reverse strings in python?". I asked chatgpt and I got the answer. But later on, a curiosity sparked in me. I opened a browser tab and searched for the same question. I got several results. I opened a result from stack overflow. To my utter surprise, I found that the expert minds here discussed a lot of approaches towards the question. Some said what is the optimal way to solve this.

Motivating example:

Q: how to reverse strings in python?



LLM generated

Stack Overflow discussion

Figure 1: Screenshot of the responses of the question

I initially got motivation from this. But later on, to have a stronger motivation, I conducted a survey. I collected 29 responses. **24** are **IIT** students and **5** are **software developers** from **Cefalo.**

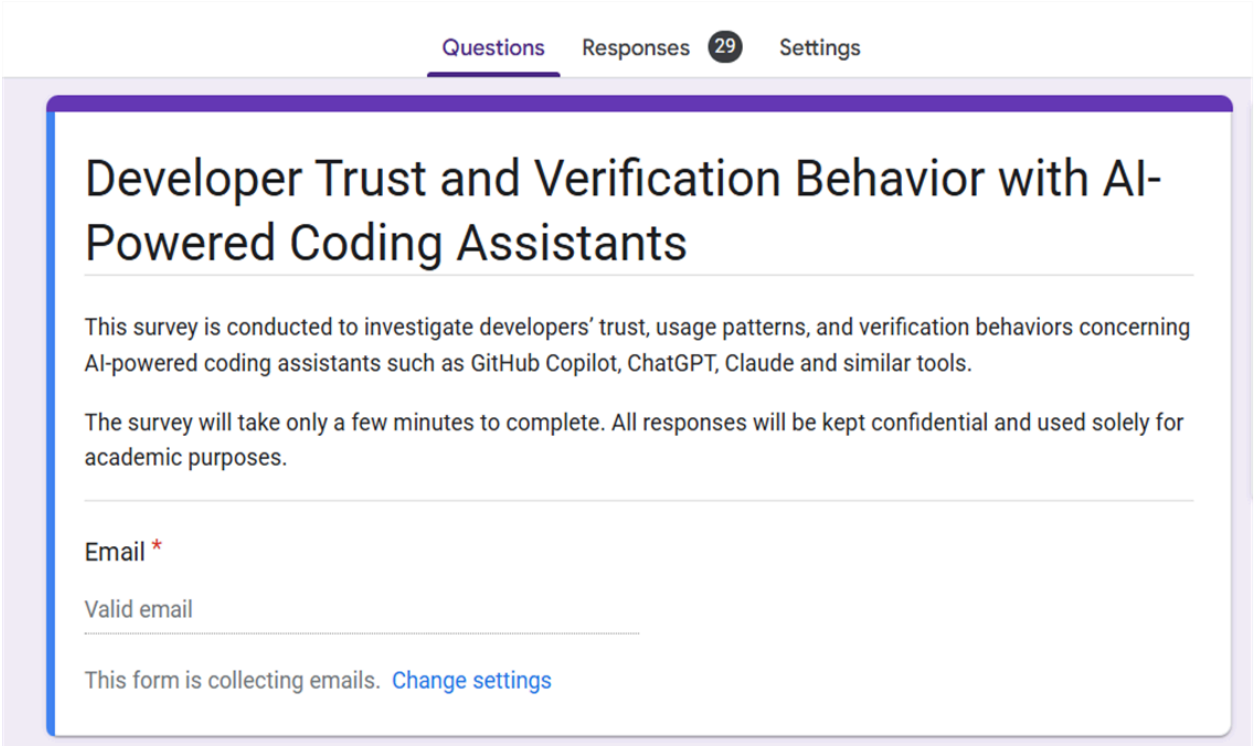


Figure 2: Survey questions

I asked them several questions. The questions and response results from the survey are given below:

Q1:

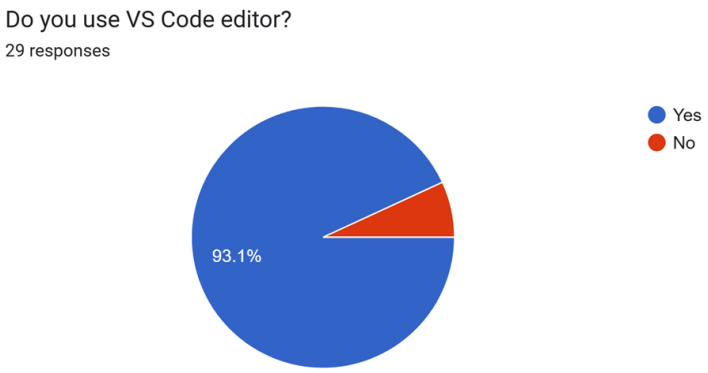


Figure 3: Survey question-1

Q2:

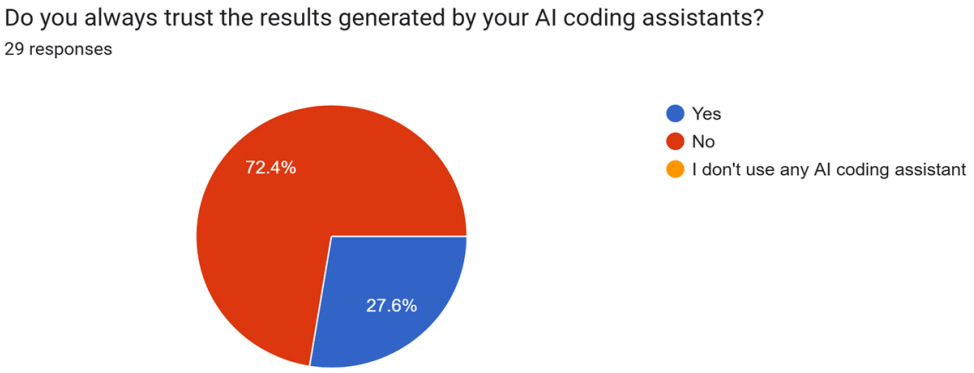


Figure 4: Survey question-2

Q3:

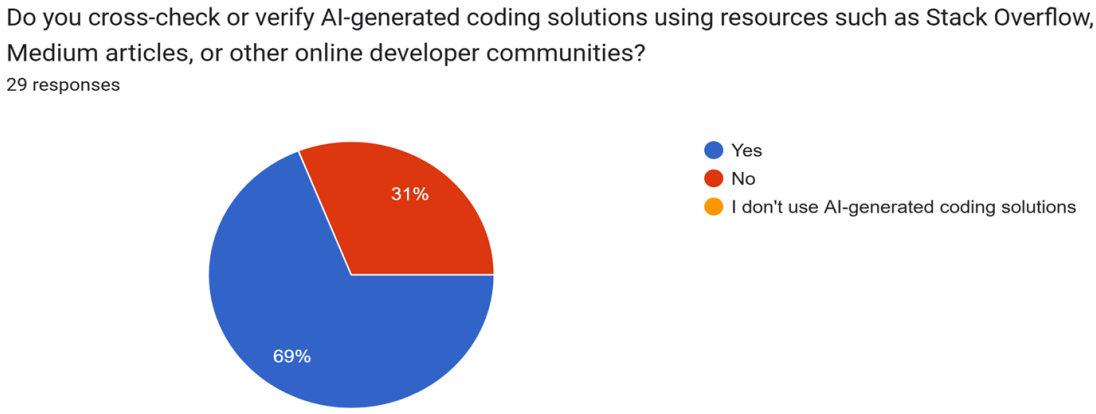


Figure 5: Survey question-3

Q4:

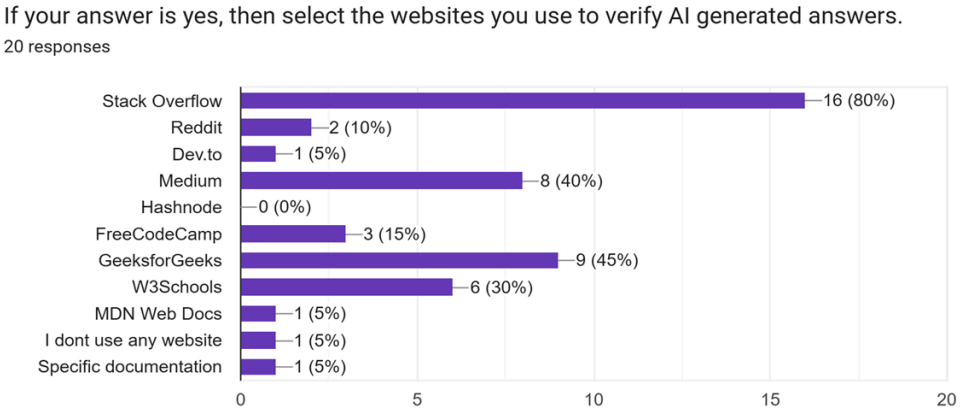


Figure 6: Survey question-4

So, we are proposing **StackRAG**, the VS Code Extension which will combine expert knowledge from Stack Overflow with Large Language Models.

1.2 Project Description

This introduces **StackRAG**, a VS Code extension to help developers solve their coding problems by using the strengths of Stack Overflow with Large Language Models. It focuses on supporting python and javascript languages. Stack Overflow has long been a trusted platform for developers to find community-verified solutions. Nowadays, with the rise of Large Language Models like ChatGPT, Gemini etc, many developers also rely on them for quick answers. However, each approach has its drawbacks. Searching through Stack Overflow threads can be time-consuming and requires filtering through multiple posts to find a clear solution. On the other hand, Large Language Models can sometimes produce answers that seem correct but are actually inaccurate or unreliable.

Our proposed tool **StackRAG** searches through Stack Overflow posts related to python and javascript languages. It fetches the accepted answers of questions and builds a question answer knowledge base. In the runtime, developers ask questions. The tool searches through the knowledge base to find the question-answer pairs relevant to the query. Then it passes the retrieved information along with the codebase context to Large Language Models to generate responses.

Broadly the features are:

1. **Interactive Chat:** Generates responses to the developer queries and attaches the links of stack overflow posts used to generate the answers.
2. **Bug fixing:** Suggests potential fixes for the developer provided coding errors.
3. **Code refactoring:** Suggests meaningful improvements to make existing code cleaner, more efficient and maintainable.
4. **Comparative analysis :** Shows the output of a developer query generated by StackRAG and VSCode Github copilot side by side in the extension UI.
5. **VS Code extension:** it will be a vscode extension.

1.3 Project Scope

The scope of **StackRAG** is mentioned below:

- It will be a VSCode extension.
- It will have general chat, bug fixing and code refactoring facilities.
- It is powered by a RAG system and LLM.
- It will provide users facility to compare the tool output with vscode github copilot interface.

1.4 Timeline

The estimated timeline for PyTypeWizard is delineated below:

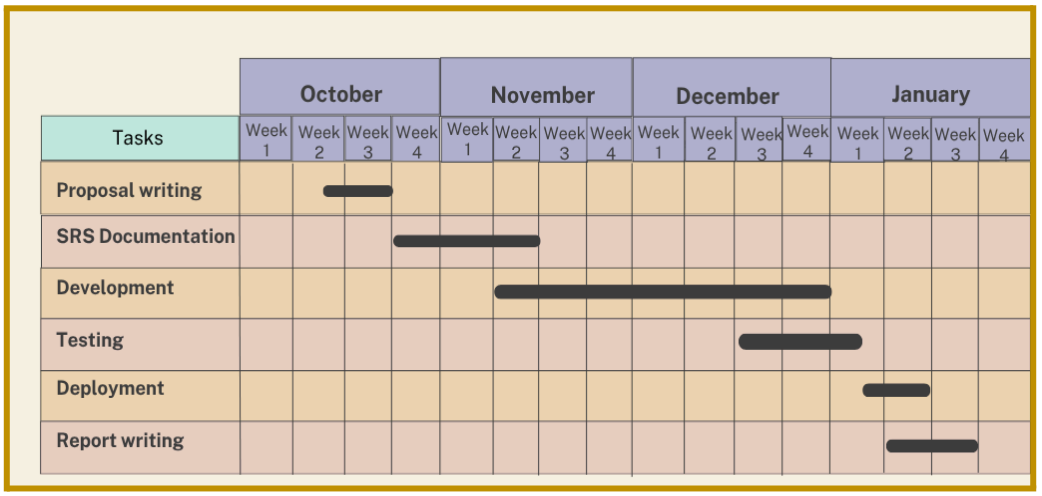


Figure 7: Project Timeline

1.5 Purpose of Document

The purposes of this document are:

- Identify and analyze the requirements
- Design the architecture
- Design the test plan
- Describe methodology
- Reduce the development effort
- Improve understanding

2. Project Description

This document outlines the Quality Function Deployment (QFD) process for StackRAG. The development of StackRAG's QFD began with an extensive review of relevant research. Their findings offered a deep understanding of the domain and informed our approach.

In addition to this research, we engaged directly with key stakeholders from the software development industry. By consulting with developers across various organizations, we gained valuable perspectives on their needs and challenges, ensuring that StackRAG addresses real-world concerns effectively.

2.1 Quality Function Deployment

Quality Function Deployment (QFD) is a technique that translates the needs of the

customer into technical requirements for the software. For this project, the following requirements are identified

2.2.1 Normal Requirements

These are fundamental features that users expect.

1. **Interactive Chat:** The tool provides interactive chat facility for developers to ask coding related questions and know the answers.
2. **Code Refactoring:** The tool provides code refactoring mode to refactor code.
3. **Bug Fixing:** The tool provides fix mode to refactor code.
4. **Integration with VSCode:** Users can access all its functionalities directly within the VSCode editor without needing to switch tools.

2.2.2 Expected Requirements

These features directly impact user satisfaction based on how well they are implemented.

1. **Context usage for generating responses:** It will use the currently open code file as a context to have consistent and accurate response generation.
2. **Appending links of related Stack Overflow Posts:** It will append the links of related Stack Overflow posts with the generated responses so that users can navigate to the links and see the posts used to generate responses.

2.2.3 Exciting Requirements

These features go beyond user expectations, providing unique value. Their presence can greatly enhance user satisfaction.

1. **Side by side view of StackRAG and VSCode github copilot response:** Providing the side by side view of the response of StackRAG and VSCode github copilot for a problem for users to compare between them.
2. **Codebase indexing:** Incorporating on demand codebase indexing by using RAG architecture to fetch relevant code context and generate accurate, context-specific responses.
3. **Codebase summary:** Providing the comprehensive summary of the codebase.

2.2 Usage Scenario

StackRAG is a VSCode extension designed to help developers solve their coding problems with the help of LLM and Stack Overflow. It provides chat facility for developers to ask coding related questions, code refactoring option, context-aware

fix suggestions for coding bugs and on demand codebase indexing for getting more context-aware responses. Along with responses, StackRAG provides the related Stack Overflow posts' links. This helps developers verify the provided solution with the Stack Overflow posts and gain more insights about the topic from the posts. The usage scenario for that particular tool is delineated below:

Installation

Users can download and install the StackRAG extension from the VSCode Marketplace.

Chat Facility to ask coding related questions

StackRAG provides a chat facility for developers to ask coding related questions. It provides answers to the questions with the help of the relevant Stack Overflow posts retrieved from knowledge base and LLM. It considers code files as context if users attach code files while asking the questions.

Potential Fix Suggestions for Bugs

StackRAG provides potential bug fixing suggestions. Users can view and apply the recommended fixes. It also explains the fix suggestions along with relevant Stack Overflow posts' links helping developers understand why the error occurred and how the fix resolves it.

Potential suggestions for Code Refactoring

StackRAG provides potential code refactoring suggestions. Users can view and apply the recommended code refactoring. It also explains the code refactoring suggestions along with relevant Stack Overflow posts' links helping developers understand how the suggestions refactor the code.

Side by side view of responses from both StackRAG and VSCode GitHub Copilot

StackRAG provides an option for users to have responses for a query generated from both the extension itself and VSCode GitHub Copilot. It helps users compare between the responses and choose one which they think is more applicable to their problem.

Codebase indexing for more context-aware responses

StackRAG provides an on demand codebase indexing facility for users to have more context-aware responses. After the initial indexing of the codebase, Users can re-sync the codebase indexing anytime.

3. Scenario-Based Modeling

Scenario-based modeling is a method used in software development and design to create models that illustrate how a system behaves or responds to different real-world situations or scenarios. These scenarios are usually based on user interactions, events, or conditions, helping developers visualize and understand how the software will function in various contexts. By simulating these scenarios, developers can identify potential issues, make informed design choices, and ensure that the software meets user needs in diverse usage situations.

3.1 Use Case Diagram

A use case diagram is a visual tool in software engineering that depicts how a system interacts with external entities or actors to accomplish specific tasks or goals. It shows the connections between use cases (which represent system functions) and actors (which represent external users or systems). Use case diagrams are typically created during the requirements gathering phase, where system functionalities and user interactions are identified and then mapped out using specialized modeling tools.

3.1.1 Level 0:

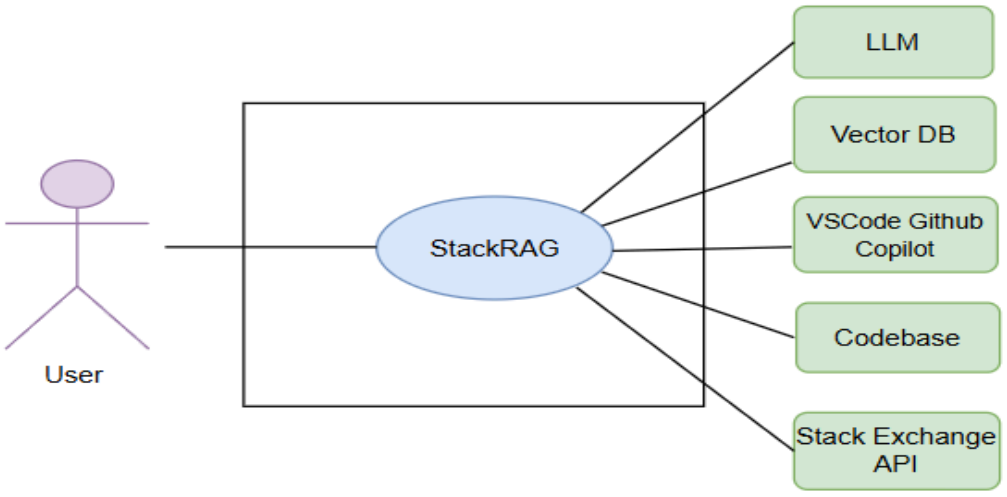


Figure 8: Level 0 (StackRAG)

Use Case Name: StackRAG System
Use Case ID: 0
Primary Actors: User
Secondary Actors: LLM, Vector DB, Codebase, VSCode Github Copilot, Stack Exchange API
Goal in Context: The diagram above illustrates the complete system overview for StackRAG, a tool designed for helping developers for code generation, refactoring and bug fixing.
Description: This is the high level view or level 0 view of the stackRAG system. It has one primary actor called user. The secondary actors are LLM, Vector DB, Codebase, VSCode Github Copilot, Stack Exchange API.

3.1.1 Level 1:

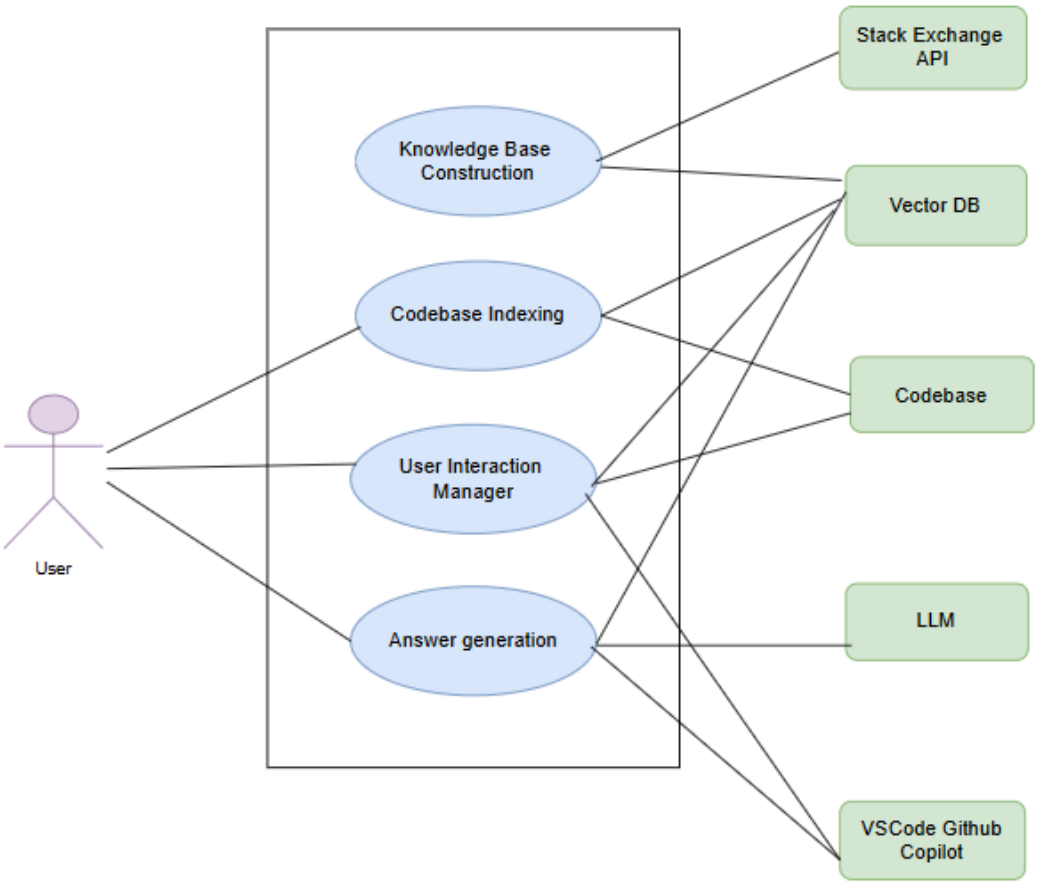


Figure 9: Level 1 (Modules of StackRAG)

Use Case Name: Modules of StackRAG

Use Case ID: 1

Primary Actors: User

Secondary Actors: LLM, Vector DB, Codebase, VSCode Github Copilot, Stack Exchange API

Goal in Context: The diagram above shows the modules of StackRAG. It consists of 5 modules which are further elaborated below.

Description:

1. User Interaction Manager

- **Functionality:** this module manages all sorts of user interaction. Like activating the extension, preparing the codebase indexing, previous interactions.
- **Purpose:** activates the extension, manages user interaction with the extension.
- **Key Role:** Acts as the starting point for all system operations.

2. Knowledge Base Construction

- **Functionality:** this module builds the knowledge base of the system. It consists of three parts: searching from stack overflow, indexing the project and building BM25 reranker.
- **Purpose:** builds the knowledge base of the system with stack overflow posts.
- **Key Dependencies:** depends on stack exchange api key.
- **Output:** we can retrieve the embedded knowledge of stack overflow.

3. Codebase Indexing

- **Functionality**: Gathers relevant code snippets or surrounding code context from the user's project. This is crucial for providing context-aware solutions. This module builds the codebase indexing.
- **Purpose**: By understanding the broader code context, this module enables more accurate and intelligent solutions.
- **Key Dependencies**: Works closely with the **Code Base** and **Vector DB** to ensure all suggestions are tailored to the specific code environment.

4. Answer Generation

- **Functionality**: this module is responsible for generating solutions.
  Solutions include: chat response, bug fixing, code refactoring.
- **Purpose:** generates the solution.
- **Key Dependencies**: depends on knowledge base, indexed codebase and llms.

3.1.1 Level 1.1:

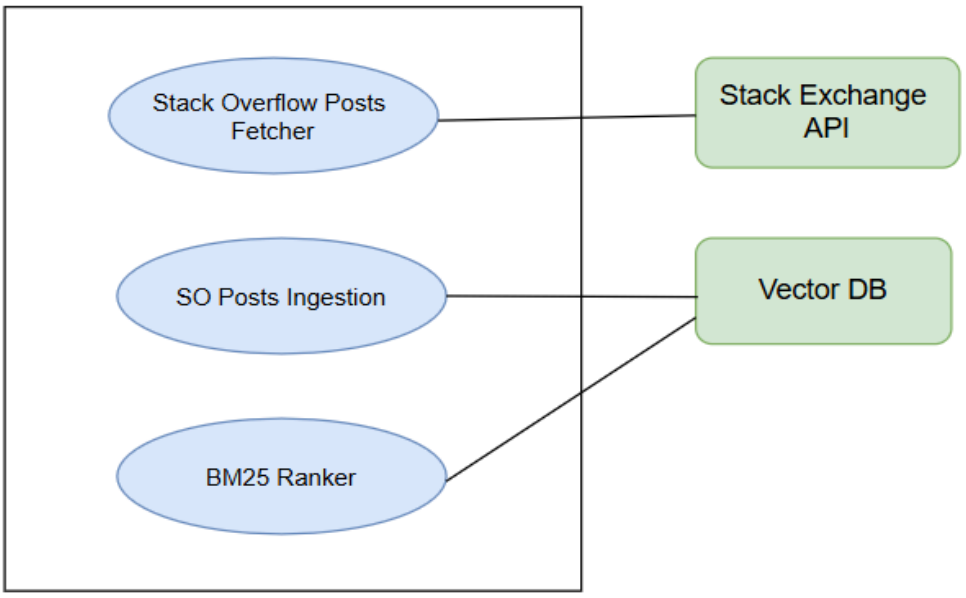


Figure 10: level 1.1 (knowledge base system)

Use Case Name: knowledge base system
Use Case ID: 2
Primary Actors: No primary actor
Secondary Actors: Vector DB, Stack Exchange API
Goal in Context: The goal of this system is to build a knowledge base.
Description:

1. **Stack Overflow Posts Fetcher:** it fetches Stack Overflow Posts by using Stack Exchange API.
2. **SO Posts Ingestion:** It ingests the fetched posts.
3. **BM25 Ranker:** it builds BM25 reranker to optimize the SO posts fetching from vector db later.

3.1.1 Level 1.2:

Use Case Name: codebase indexing system
Use Case ID: 3
Primary Actors: User
Secondary Actors: Vector DB, Codebase
Goal in Context: indexes open codebase on demand.

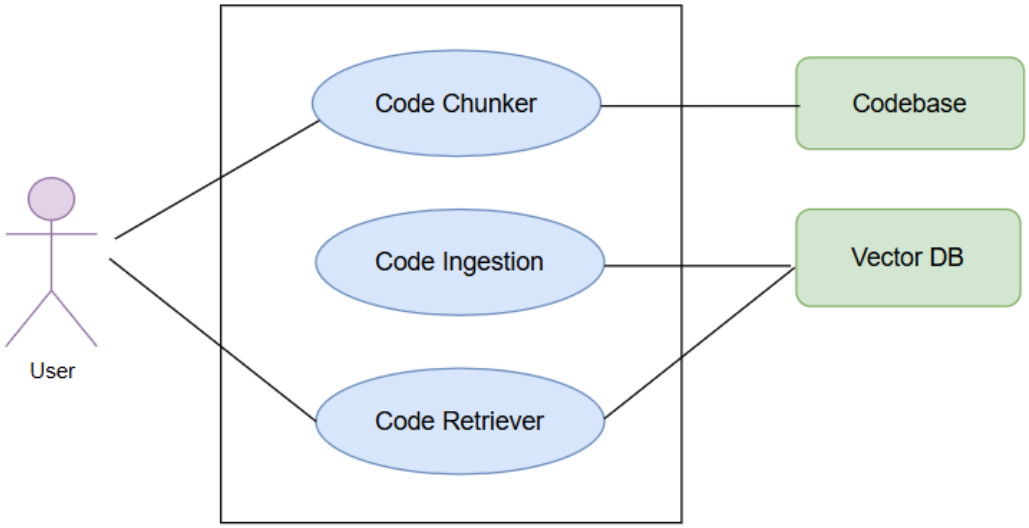


Figure 11: level 1.2 (codebase indexing system)

Description:

1. **Code Chunker:** it chunks the py and js code files based on statement and function types.
2. **Code Ingestion:** it ingests the chunked codes into vector db.
3. **Code Retriever:** the retriever part retrieves the indexed code from the vector db.

3.1.1 Level 1.4:

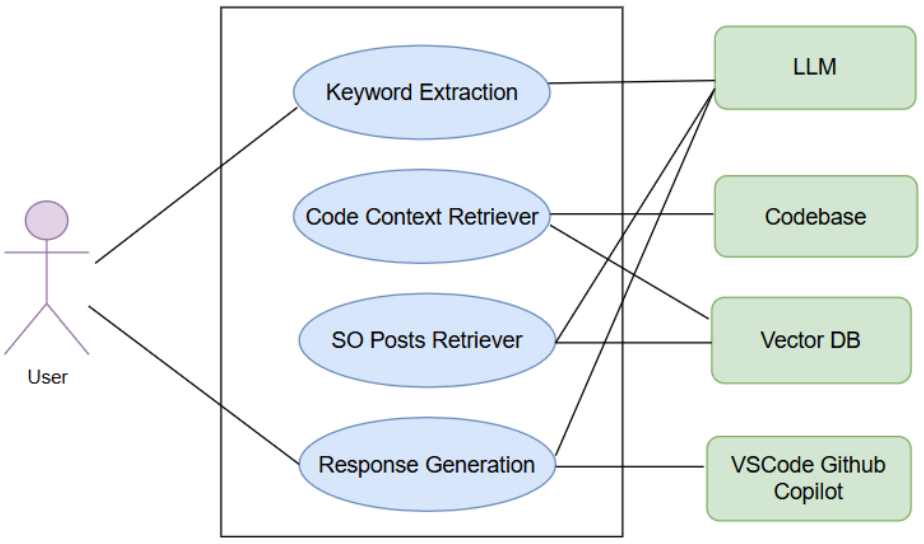


Figure 12: level 1.4 (answer generation system)

Use Case Name: Answer Generation System
Use Case ID: 4
Primary Actors: User
Secondary Actors: LLM, Vector DB, Codebase, VSCode Github Copilot
Goal in Context: it is responsible for generating response.
Description:

1. **Keyword Extraction:** it extracts keywords from the given user query.
2. **Code Context Retriever:** it retrieves the relevant codes from vector db.
3. **SO Posts Retriever:** it retrieves the relevant SO posts from vector db.
4. **Response Generation:** it generates responses using retrieved relevant information with the llm.

3.1.1 Level 1.4.3:

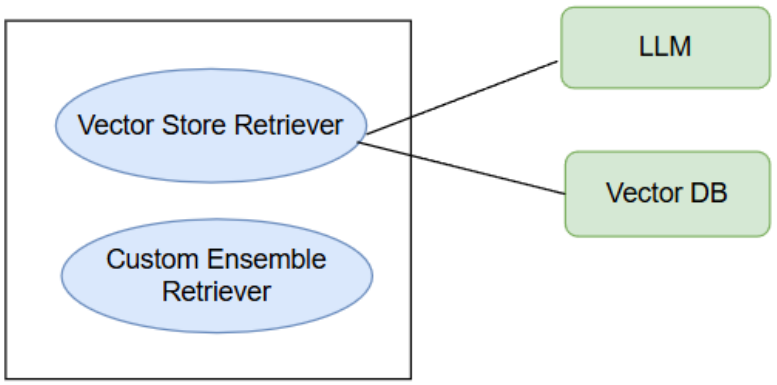


Figure 13: level 1.4.3 (SO posts retriever system)

Use Case Name: SO posts retriever system
Use Case ID: 5
Primary Actors: User
Secondary Actors: LLM, Vector DB
Goal in Context: it is responsible for retrieving SO posts efficiently from vector db.
Description:

1. **Vector Store Retriever:** it retrieves SO posts using vector store retriever. This retriever retrieves SO posts from vector db.
2. **Custom Ensemble Retriever:** it achieves similar performance like langchain's previous vector store retriever. As it is now deprecated, the custom ensemble retriever performs the similar task.

4. Data Based Modeling

No relational database system is used for this tool. Only VectorDB is used to store the required structured collection. Below is the structure of those collection:

Documents	**Attributes**	**Types**
Question	page_content	TEXT
	Type (metadata)	TEXT
	Link (metadata)	TEXT
	Score (metadata)	INTEGER
	Tags (metadata)	ARRAY[VARCHAR(50)]
	Question_id (metadata)	INTEGER
	Answer_count (metadata)	INTEGER
	View-count (metadata)	INTEGER
Answer	page_content	TEXT
	Type (metadata)	TEXT
	Question_id (metadata)	INTEGER
	Score (metadata)	INTEGER
Code	page_content	TEXT
	filePath (metadata)	VARCHAR(255)
	Type (metadata)	VARCHAR(30)
	Extension (metadata)	VARCHAR(10)

Table-1: Schema table

5. Class-Based Modeling

Class-based modeling is a visual representation in software engineering that illustrates the structure of a software system using classes, their attributes, and relationships. It is commonly used in object-oriented design and modeling.

5.1 Analysis Classes

After identifying nouns from the scenario, I filtered nouns belonging to the solution domain using General Classification (External entities, Things, Events, Roles, Organizational units, Places, and Structures). Nouns selected as a potential class were filtered using Selection Criteria (Retained information, Needed services, Multiple attributes, Common attributes, Common operations, and Essential requirements). After performing analysis on potential classes, I have found the following analysis classes:

1. Models
2. keywordExtractor
3. Retriever
4. CustomEnsembleRetriever
5. Chat
6. AnswerGeneration
7. Ingestion
8. BuildAndSaveBM25
9. JSChunker
10. PYChunker
11. CodeIngestion
12. CodebaseIndexManager
13. VSCodeExtensionManager

5.2 Class Cards

Models	
Attributes	**Methods**
- ollama_embeddings - ollama_chat - mistral_embeddings - mistral_chat	+ __init__()
Responsibilities	**Collaborators**
Define the chat and embedding models	• AnswerGeneration • Retriever

Table-2: CRC card (**Models)**

KeywordExtractor

Attributes	Methods
- keyword_extractor_prompt - extracted_keywords - keyword_chain	+ extract_keywords()
Responsibilities	**Collaborators**
Extract the keywords using the keyword extracting prompt.	• AnswerGeneration

Table-3: CRC card (**KeywordExtractor)**

Retriever	
Attributes	**Methods**
- bm25_retriever - vector_retriever - ensemble_retriever - questions - so_discussions	+ retrieve()
Responsibilities	**Collaborators**
Retrieves related SO posts using both bm25 and vector retriever.	• CustomEnsembleRetriever • AnswerGeneration • BM25Builder • Models

Table-4: CRC card (**Retriever)**

CustomEnsembleRetriever	
Attributes	**Methods**
- retrievers - weights	+ __init__() + get_relevant_documents()
Responsibilities	**Collaborators**
Define a custom ensemble retriever	• Retriever

Table-5: CRC card (**CustomEnsembleRetriever)**

AnswerGeneration

Attributes	Methods
- answer_prompt - answer_chain - response	+ generate_chat_response() + fix_bug() + refactor_code()
Responsibilities	**Collaborators**
Generates responses to users' requests.	• Models • Retriever • Keyword Extractor • VSCodeExtensionManager

Table-6: CRC card(**AnswerGeneration)**

SOPostsFetcher	
Attributes	**Methods**
- state - page - accepted_qs - accepted-answer_ids - answers - qna_pairs	+ fetch_all_questions_with_accepted_answers() + fetch_all_questions_with_no_accepted_answers() + clean_html_content() + load_state() + save_state() + save_questions() + chunk_list()
Responsibilities	**Collaborators**
Fetches posts from Stack Overflow	• Ingestion

Table-7: CRC card(**SOPostsFetcher)**

Ingestion	
Attributes	**Methods**
- text_splitter - embeddings - qa_collection - qa_threads - SO_collections - documents	+ main() + thread_to_document() + process_documents() + store_in_vectordb()

Responsibilities	Collaborators
Ingest the fetched SO posts into the vector database.	• SOPostsFetcher • BM25Builder

Table-8: CRC card(**Ingestion)**

BM25Builder	
Attributes	**Methods**
- raw_collection - all_docs - question_docs	+build_and_save_bm25()
Responsibilities	**Collaborators**
Build and save the bm25 reranker for future use.	• Retriever • Ingestion

Table-9: CRC card(**BuildAndSaveBM25)**

JSChunker	
Attributes	**Methods**
- workspace_path - jsFiles - chunks	+ findJsFiles() + splitJsCodes() + readJsFilesConcurrently() + processJsFiles()
Responsibilities	**Collaborators**
Chunks the javascript codes into function, statement level.	• CodebaseIndexManager • CodeIngestion

Table-10: CRC card(**JSChunker)**

PYChunker	
Attributes	**Methods**
- workspace_path - pythonFiles - chunks	+ findPythonFiles() + splitPythonCodes() + readPythonFilesConcurrently() + processPythonFiles()

Responsibilities	Collaborators
Chunks the python codes into function, statement level.	• CodebaseIndexManager • CodeIngestion

Table-11: CRC card(**PYChunker)**

CodeIngestion	
Attributes	**Methods**
- text_splitter - embeddings - collection - threads - code_collections - documents	+ main() + thread_to_document() + process_documents() + store_in_vectordb()
Responsibilities	**Collaborators**
Ingest chunked codes into a vector database.	• CodebaseIndexManager • JSChunker • PYChunker

Table-12: CRC card(**CodeIngestion)**

CodebaseIndexManager	
Attributes	**Methods**
- embeddingModel - collectionName	+ readIndex() + createIndex() + updateIndex() + deleteIndex()
Responsibilities	**Collaborators**
Manages the crud operations related to codebase index.	• CodebaseIndexManager • JSChunker • PYChunker • VSCodeExtensionManager

Table-13: CRC card(**CodebaseIndexManager)**

VSCodeExtensionManager	
Attributes	**Methods**

- webviewView	+ activate_extension() + showInwebview() + getWebviewContent() + callCopilot()
Responsibilities	**Collaborators**
Manage the user interface within VSCode, including displaying chatbox, options for bug fixing, code refactoring, history, codebase indexing.	• CodebaseIndexManager • AnswerGeneration

Table-14: CRC card(**VSCodeExtensionManager)**

5.3 CRC Diagram

The CRC diagram of the classes is given below:

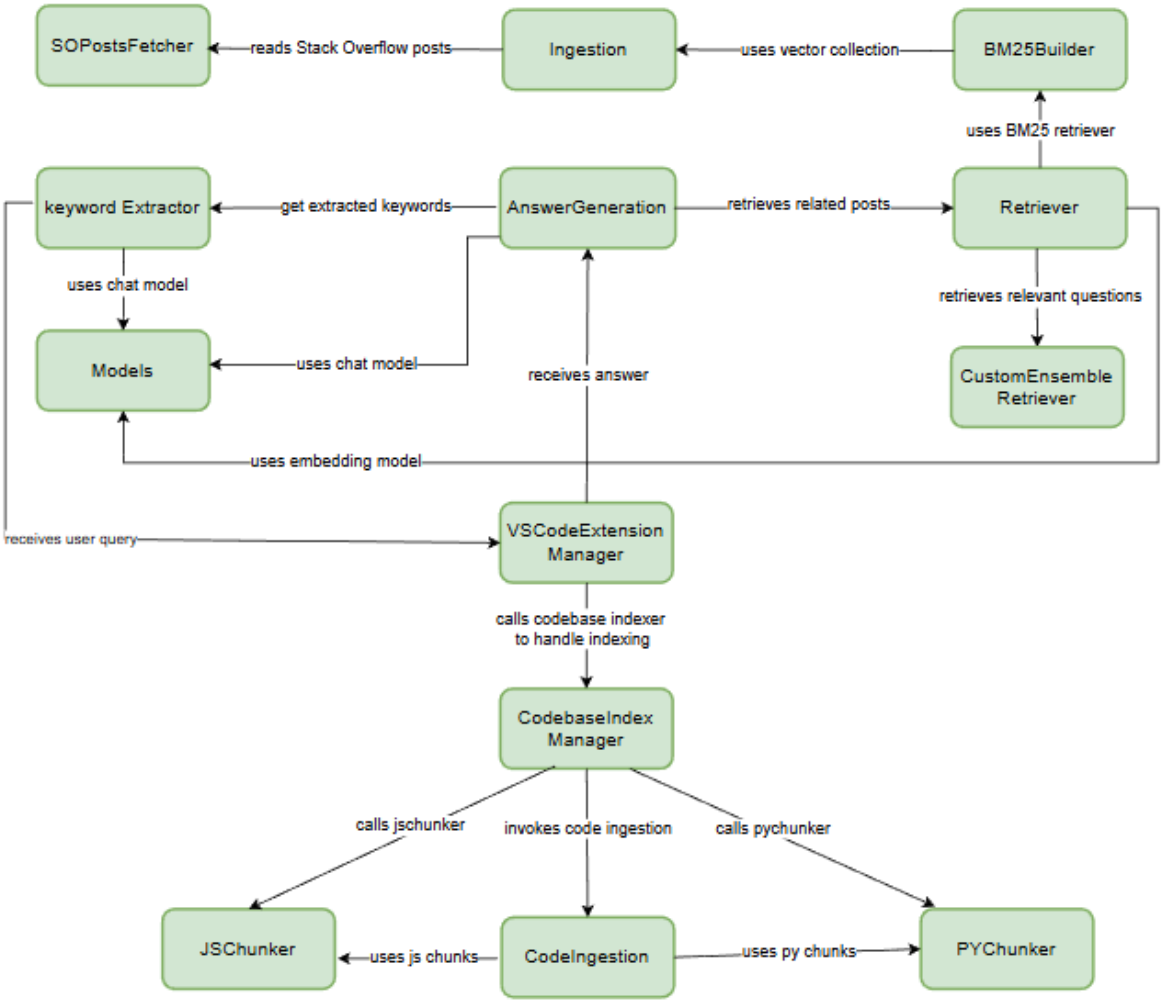


Figure 14: CRC Diagram of StackRAG

6. Architectural Design

Architectural design is a visual representation in software engineering that outlines the high-level structure and organization of a software system. It focuses on defining the major components or modules of the system, their interactions, and the overall system's architecture.

6.1 Architectural Context Diagram

At the architectural design level, a software architect uses an architectural context diagram (ACD) to model how software interacts with entities external to its boundaries. Systems that interoperate with the target system are represented as -

- **Superordinate systems**— Those systems that use the target system as part of some higher-level processing scheme.
- **Subordinate systems**— Those systems that are used by the target system and provide data or processing that are necessary to complete target system functionality.
- **Peer-level systems—** Those systems that interact on a peer-to-peer basis (i.e., information is either produced or consumed by the peers and the target system)
- **Actors—** Entities (people, devices) that interact with the target system by producing or consuming information that is necessary for requisite processing.

Each of these external entities communicates with the target system through an Interface. Representation of StackRAGin Architectural Context Diagram is delineated as follows:

- **Superordinate systems** - Our system does not have any superordinate system.
- **Subordinate systems -** Our system uses the ***LLM*** to generate answers, ***Stack Overflow Posts*** as an external knowledge source, ***Codebase*** for context and ***Vector DB*** for storing data.
- **Peer-level systems -** Our system has one peer-level system that is ***VSCode Github Copilot***.
- **Actors - *Developers*** are the core users of this system.

The architectural context diagram for the tool is given below.

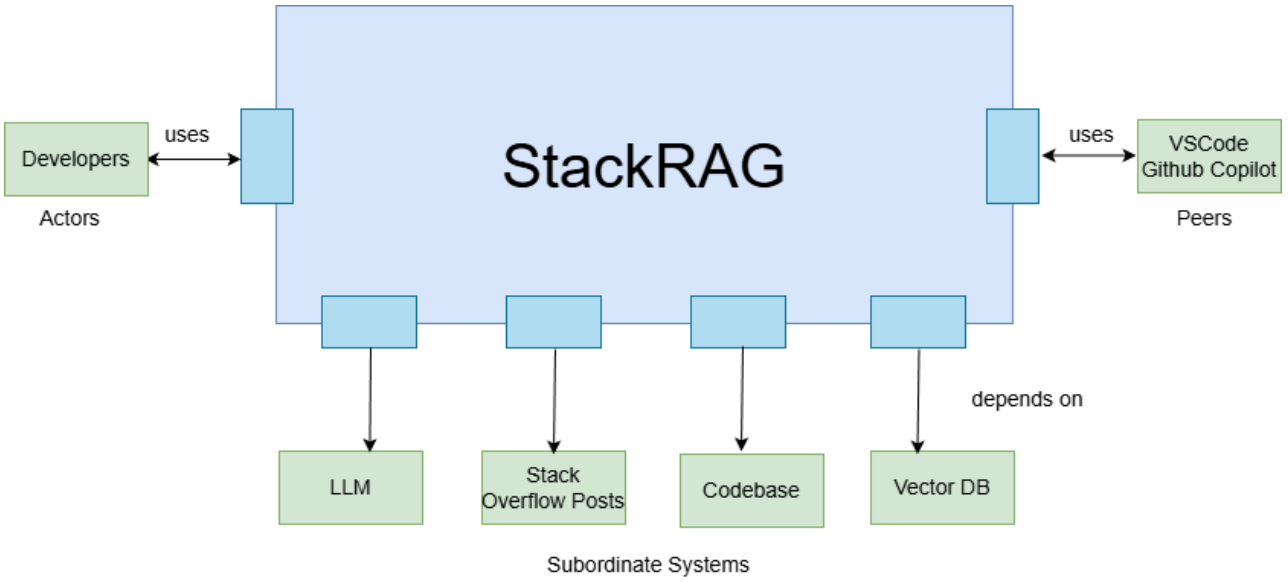


Figure 15: Architectural Context Diagram

6.2 Archetypes

The layered architecture:

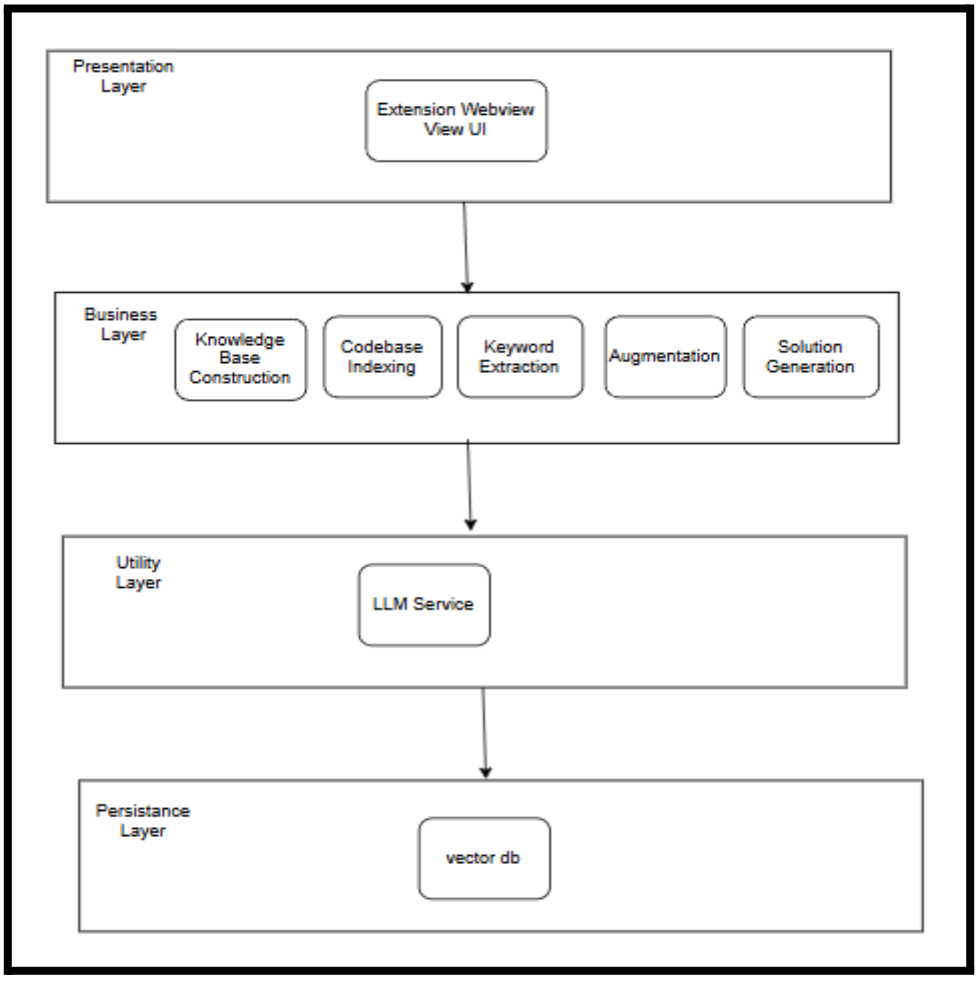


Figure 16: Layered Architecture

Description:

1. Presentation Layer
    - Extension Wevview View UI: it handles the GUI part of the tool.
2. Business layer
    - Knowledge base construction: it builds the knowledge base for the system.
    - Codebase indexing: it indexes the codebase on demand.
    - Keyword extraction: it extracts the keyword from the user query.
    - Augmentation: it augments the retrieved relevant information with the user query.
    - Solution generation: it is responsible for generating the solution using llm.
3. Utility layer
    - Llm service: the tool uses llm service as a utility to generate the final solution.
4. Persistence layer
    - Vector db: it stores our knowledge base and codebase indexing.

The archetypes for the tool are defined below.

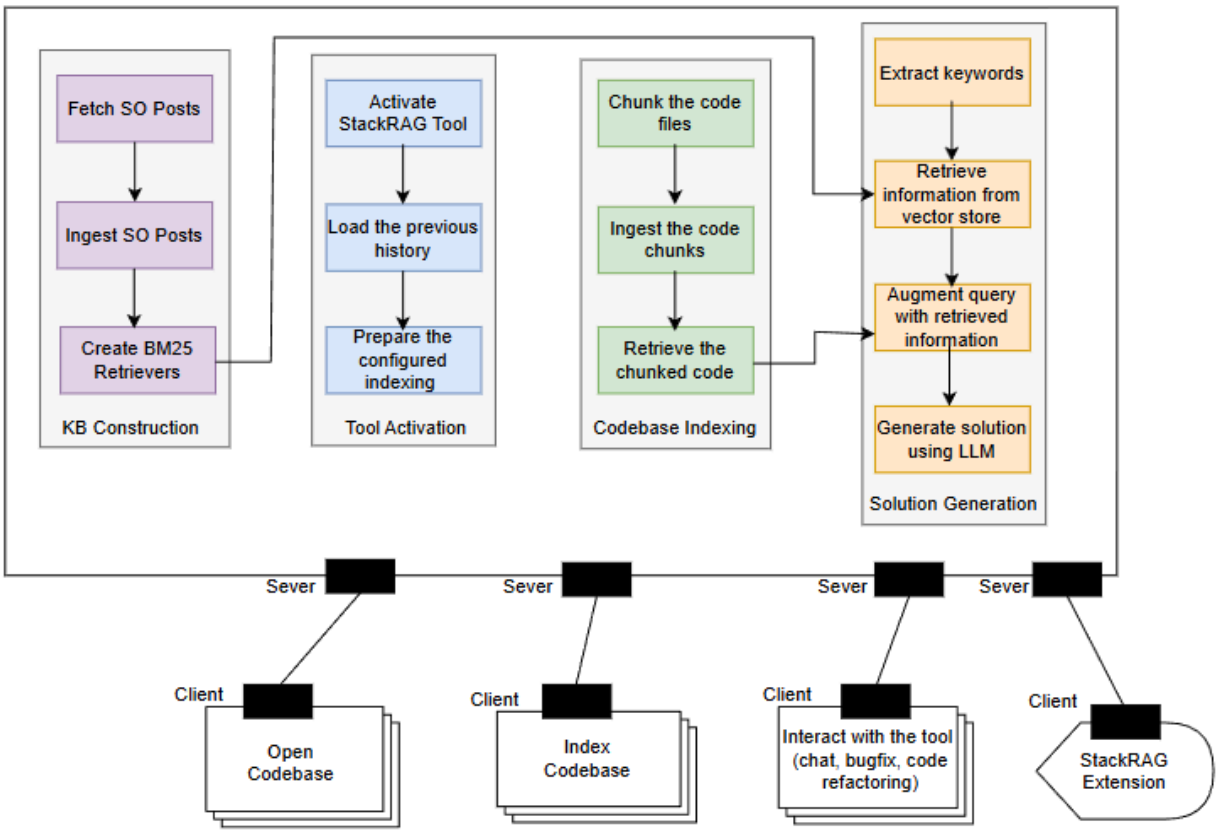


Figure 17: Archetypes

This client-server architecture is tailored to balance real-time performance with advanced processing capabilities.

KB Construction (Server Side)

- **Fetch SO posts:** it fetches posts from stack overflow website using stack exchange api.
- **Ingest SO posts:** it ingests the fetched posts and stores into vector db.
- **Create BM25 retrievers:** it creates a BM25 retriever for optimization purposes.

Tool Activation (Client Side)

The clients open a codebase in vscode.

Tool Activation (Server Side)

- **Activate StackRAG Tool**: Triggers the StackRAG upon opening a Python or Javascript project in VSCode.
- **load the previous history:** it loads the previous interaction history from the json file.
- **prepares codebase indexing:** it loads the codebase indexing information from the json file where it stores the state.

Codebase Indexing (client Side)

The clients may want to index codebase on demand.

Codebase Indexing (Server Side)

- **chunk the code files**: it chunks the code files (python and javascript).
- **index the chunked code**: it indexes the chunked code portions.
- **Retrieve the chunked code**: Retrieves relevant parts of the codebase based on the user query.

Solution Generation (Client Side)

The clients want to interact with the system. The interaction takes the form:
interaction chat, bug fixing, code refactoring.

Solution Generation (Server Side)

- **Extract Keyword**: it extracts keywords from the user query.
- **Retrieve information from vector store**: it retrieves the relevant information from the vector db. It uses the BM25 retriever in this part.
- **Augment query with retrieved information**: it augments the retrieved information with the user query.
- **Generate Solution with LLM**: it generates the solution to the user query using llm.

6.3 Top Level Components

The overall architectural structure with top-level components is illustrated below

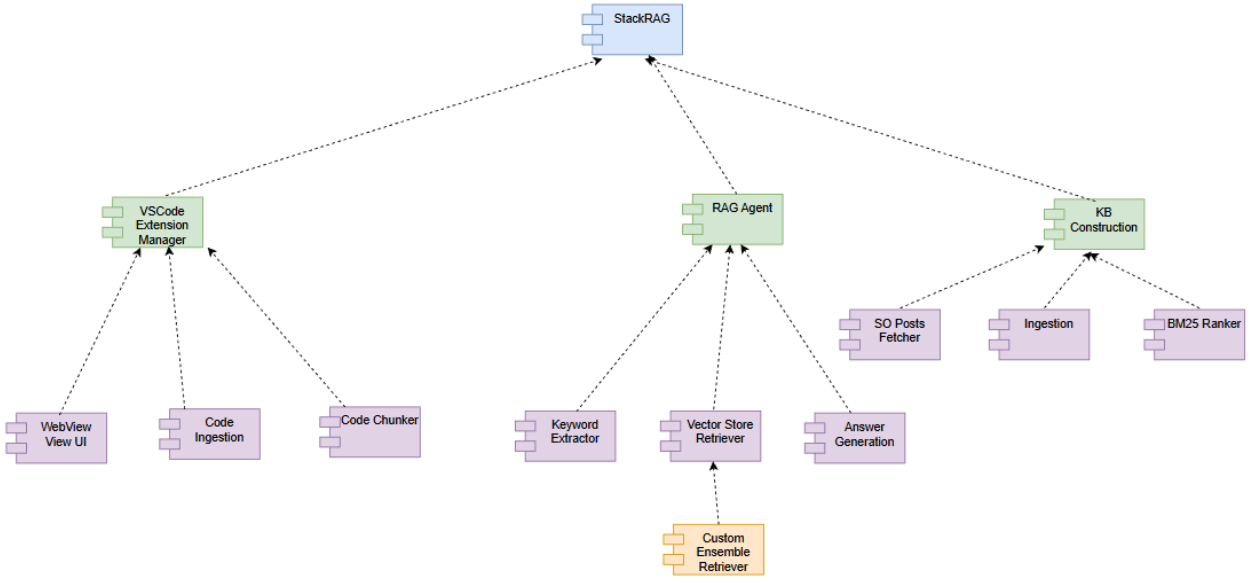


Figure 18: Top Level Components

6.4 Mapping Requirements to Software Architecture

The mapping of the derived components with the requirements can be shown below:

Requirements	**Components**
Integration into VSCode	Webview View UI
Interactive chat	BM25 Ranker,Keyword Extractor, Vector Store Retriever, Custom Ensemble Retriever, Answer Generation
Bug fixing	BM25 Ranker,Keyword Extractor, Vector Store Retriever, Custom Ensemble Retriever, Answer Generation
Code refactoring	BM25 Ranker,Keyword Extractor, Vector Store Retriever, Custom Ensemble Retriever, Answer Generation
Codebase indexing	Code Ingestion, Code Chunker, Codebase Indexer
Codebase Summary	Code Ingestion, Code Chunker, Codebase Indexer
Appending links of related Stack Overflow Posts	Vector Store Retriever, Custom Ensemble Retriever

Side by side view of StackRAG and VSCode github copilot response	Webview View UI

Table-15: Requirements Mapping with components

6.5 Deployment Diagram

To effectively deploy StackRAG, it's essential to have a clear and straightforward deployment architecture. This ensures scalability, maintainability, and ease of implementation. Below is a simplified deployment diagram partitioned into Frontend and Backend sections, along with External Services.

The deployment diagram of the StackRAG is shown below:

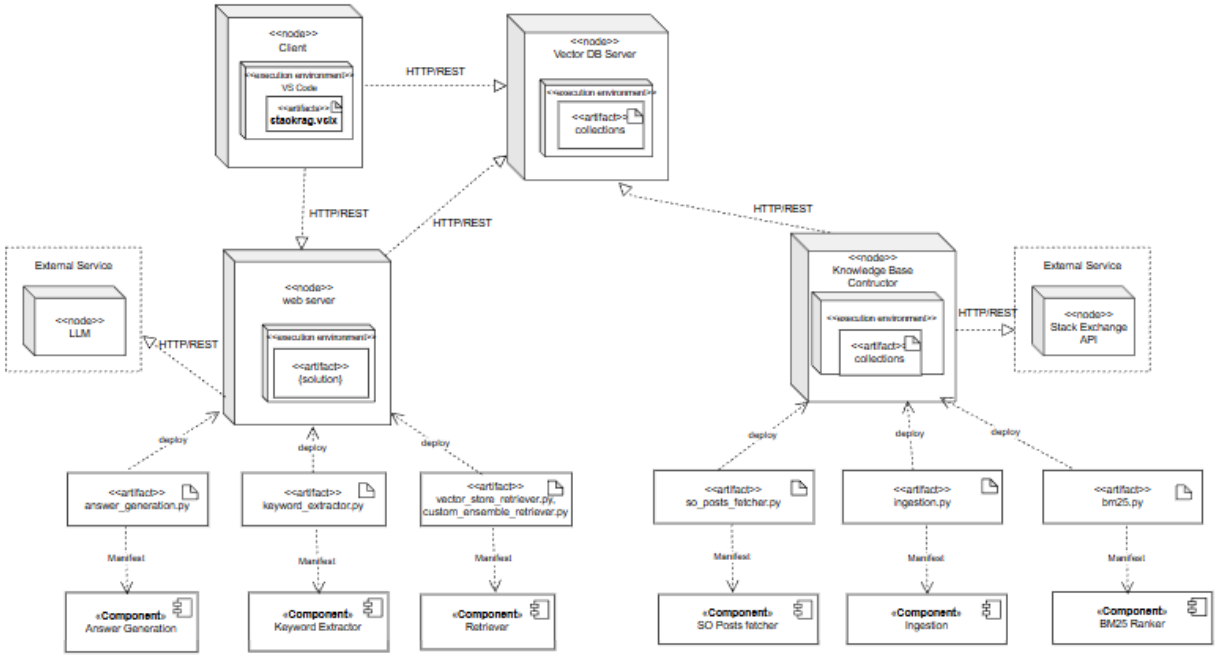


Figure 19: Deployment diagram

7. Preliminary Test Plan

In this chapter, a high-level description of testing goals and a summary of features to be tested are presented.

7.1 High-Level Description of Testing Goals

Testing goals for this tool are represented below from a high level-

- To verify that StackRAG meets its most important functional and

non-functional requirements.

- To ensure type-related error detection in real time is correct and reliable.
- To ensure such type hint suggestions are contextually valid to provide better code quality.
- To reduce false positives and false negatives in error detection and fix suggestions.
- To confirm that the tool generates detailed and understable error explanations.
- To test that user feedback on suggestions is collected, processed, and used effectively to enhance the tool.
- To guarantee seamless integration and smooth operation within the VSCode environment.

7.2 Test Cases

Test cases for the testing of this tool are documented below:

Test Case Id	T1
Title	Installation and Activation
Test Scenario	- Install the StackRAG extension in a fresh VSCode installation. - Activate StackRAG.
Expected Outcome	The extension installs without errors & activates successfully.

Table-16: Test Plan -1

Test Case Id	T2
Title	Chat Interaction
Test Scenario	- Ask a python related question. (e.g., how to write tuple in python) - Check that StackRAG generates output.
Expected Outcome	- StackRAG generates output aligned with the question and Stack Overflow posts.

Table-17: Test Plan -2

Test Case Id	T3
Title	Potential Solutions to Coding Error

Test Scenario	- Request to Fix a coding error (e.g., a recursion code where the base condition is wrong) - Apply the suggested fix provided by StackRAG.
Expected Outcome	The suggested fix accurately resolves the coding error.

Table-18: Test Plan -3

Test Case Id	T4
Title	Suggested Refactoring to Code
Test Scenario	- Request to refactor a code with poor structure (e.g., repeatedly performing summation of 10 numbers one by one in separate lines instead of using loop) - Apply the suggested refactoring provided by StackRAG.
Expected Outcome	The suggested refactoring provides better structure while preserving functionality.

Table-19: Test Plan -4

Test Case Id	T5
Title	Context-Aware Solutions
Test Scenario	- Attach a separate file as context. - Request to fix a bug in the attached code file.
Expected Outcome	StackRAG suggests a potential fix based on the attached context file.

Table-20: Test Plan -5

Test Case Id	T6
Title	Integration with LLM
Test Scenario	- Ask a question which might not be in the KB - Ensure that the system generates output
Expected Outcome	- StackRAG provides a solution generated by the LLM. -The explanation includes that there are no related posts of SO in the knowledge base. But llm

	generates the solution based on its own knowledge.

Table-21: Test Plan -6

Test Case Id	T7
Title	KB of Stack Overflow Posts
Test Scenario	- Ask a question on python or javascript language. (e.g., how to write “use strict” in js code)
Expected Outcome	- StackRAG provides a solution with the related SO posts links.

Table-22: Test Plan -7

Test Case Id	T8
Title	Codebase Indexing
Test Scenario	Index the open codebase. Each code chunk is of two types of categories: function and statement.
Expected Outcome	- generates a codebase indexing for the current open codebase.

Table-23: Test Plan -8

Test Case Id	T9
Title	Check Codebase Context Extraction Capabilities
Test Scenario	Check that codebase chunks are extracted to generate solutions.
Expected Outcome	- Relevant code context is extracted almost accurately, capturing necessary information for solution generation.

Table-24: Test Plan -9

Test Case Id	T10
Title	Side by Side output of stack rag and github copilot.

Test Scenario	- Ask a question related to a python code. - Select the option for side by side output.
Expected Outcome	The system shows us output in the side by side view from where we can select one output to continue.

Table-25: Test Plan -10

Test Case Id	T11
Title	Codebase summary
Test Scenario	- Ask to generate a summary for the currently open codebase.
Expected Outcome	The system gives us a summary of the currently open codebase by using the codebase indexing.

Table-26: Test Plan -11

Test Case Id	T12
Title	Reindexing the codebase
Test Scenario	- Reindex the codebase.
Expected Outcome	StackRAG refreshes the stale indexing.

Table-27: Test Plan -12

Conclusion

In the current report, I have detailed all the necessary technical specifications required to develop the **StackRAG**: A LLM and RAG powered AI Coding Assistant AS VS Code Extension. I have elaborated on the project scope, requirements, and usage scenarios in detail. Scenario-based modeling, data modeling and class-based modeling were done to establish a resilient system architecture. In addition, a general overview of the goals of testing as well as a summary of the features to be tested, has been presented. Diagrams and tables have been incorporated to ensure a thorough understanding of the technical aspects of this tool.

While StackRAG can handle chat interaction, bug fixing, code refactoring, It might be further improved in many ways: extending it to work with different IDEs and continuing to use more advanced RAG techniques to improve the solution based

on context.

This document should give a full understanding of the workflow and technical design of this tool. I hope it will be a helpful resource to gain insight into StackRAG and its development process.

Reference

[1] Davit Abrahamyan, Fatemeh H. Fard. "StackRAG Agent: Improving Developer Answers with Retrieval-Augmented Generation." 40th International Conference on Software Maintenance and Evolution (ICSME), 2024. Link

[2] Elijah Mansur, Johnson Chen, Muhammad Anas Raza, Mohammad Wardat. "RAGFix: Enhancing LLM Code Repair Using RAG and Stack Overflow Posts." 2024 IEEE International Conference on Big Data (BigData). Link

[3] Tasmia Zerin, Moumita Asad, B M Mainul Hossain, Kazi Sakib. “Repairing Responsive Layout Failures Using Retrieval Augmented Generation” in Proceedings of the 41st International Conference on Software Maintenance and Evolution (ICSME), 2025. Link

[4] S. Kabir, D. N. Udo-Imeh, B. Kou, and T. Zhang, “Is stack overflow obsolete? an empirical study of the characteristics of chatgpt answers to stack overflow questions,” in Proceedings of the 2024 CHI Conference on Human Factors in Computing Systems, 2024. Link

[5] google survey form